A determinação correta dos contextos de domínio é o passo mais crítico na migração de um monólito para microsserviços. A abordagem padrão e mais sólida da indústria para isso é o **Domain-Driven Design Estratégico (DDD)**, complementado por técnicas de análise de arquitetura reversa.

A seguir estão as etapas e práticas essenciais para identificar, isolar e desenhar esses contextos:

---

1. Descoberta de Domínio com Event Storming

Antes de olhar para o código legado, entenda o negócio. O monólito frequentemente reflete atalhos técnicos que obscurecem a verdadeira intenção do domínio.

- **Conduza sessões de Event Storming:** Reúna especialistas de negócio (*domain experts*), desenvolvedores e arquitetos para mapear a linha do tempo do fluxo de valor através de **Eventos de Domínio** (fatos ocorridos no passado, como `PedidoCriado`, `PagamentoConfirmado`, `NotaFiscalEmitida`).
- **Identifique Comandos e Gatilhos:** O que causa cada evento? (Ações de usuários, eventos temporais, integrações externas).
- **Agrupe em Agregados e Políticas:** Identifique regras de negócio e limites de consistência transacional imediata.

---

2. Identificação de Bounded Contexts (Contextos Delimitados)

Um **Bounded Context** é a fronteira lógica e linguística onde um modelo de domínio se aplica sem ambiguidade.

- **Busque a Linguagem Ubíqua (*Ubiquitous Language*):** O maior sinal de múltiplos contextos coexistindo no mesmo modelo é a **homonímia** (a mesma palavra significando coisas diferentes para setores diferentes):
  - Exemplo: O termo `Usuário` ou `Cliente`:
    - No contexto de **Autenticação**, `Usuário` é credenciais, perfil e roles.
    - No contexto de **Faturamento**, `Cliente` é CPF/CNPJ, endereço fiscal e método de pagamento.
    - No contexto de **Logística/Entrega**, `Destinatário` é coordenadas de entrega e histórico de recebimento.
  - Se uma mesma entidade no monólito possui dezenas de colunas e serve a áreas distintas da empresa, ela deve ser dividida entre diferentes Bounded Contexts.
- **Mapeie Subdomínios:**
  - **Core Domain:** O coração do negócio e diferencial competitivo (ex.: algoritmo de recomendação, motor de cálculo financeiro próprio). Merece investimento e código sob medida.
  - **Supporting Domain:** Complementa o negócio principal, mas não é um diferencial (ex.: catálogo de produtos, controle de estoque básico).
  - **Generic Domain:** Problemas comuns de mercado que poderiam ser resolvidos com SaaS ou bibliotecas prontas (ex.: autenticação/SSO, envio de notificações, gateway de pagamento).

---

3. Análise Reversa do Monólito (Mapeamento Código/Dados)

Com o modelo mental de negócio desenhado, confronte-o com a realidade do código existente:

- **Mapeamento de Entidades e Módulos:**
  - Analise o schema do banco de dados relacional. Tabelas muito conectadas por chaves estrangeiras indicam alto acoplamento de dados.
  - Agrupe tabelas por afinidade funcional.
- **Matriz de Interação CRUD:**
  - Mapeie quais controllers/serviços lêem e escrevem em quais tabelas.
  - Se o módulo de "Vendas" faz escrita direta na tabela de "Estoque", existe uma fronteira de contexto violada que precisará se transformar em comunicação via API ou mensageria assíncrona.
- **Histórico de Commits e Dependências (Code Churn):**
  - Ferramentas de análise estática e histórico de Git ajudam a revelar arquivos que mudam juntos constantemente (*temporal coupling*), indicando coesão implícita ou violação de limites.

---

4. Mapeamento de Contexto (Context Mapping)

Nem todos os microsserviços se relacionam da mesma forma. O DDD define padrões de integração que ajudam a prever como os novos serviços vão interagir com o monólito ainda vivo e entre si:

- **Shared Kernel:** Compartilhamento de dados/código (evite ao máximo em microsserviços para não criar acoplamento implícito).
- **Customer/Supplier ou Upstream/Downstream:** Um serviço depende das alterações e prazos do outro.
- **Anti-Corruption Layer (ACL):** Camada de tradução essencial durante a migração. O novo microsserviço cria seu próprio modelo limpo e se comunica com o monólito por meio de um adaptador que traduz as respostas antigas sem "contaminar" a nova arquitetura.
- **Open Host Service / Published Language:** APIs públicas e contratos bem definidos (ex.: via OpenAPI/Swagger ou Protocol Buffers) com mensageria assíncrona (RabbitMQ, Kafka, SQS).

---

5. Heurísticas para Evitar Armadilhas Comuns

Ao decidir o tamanho e escopo do contexto/serviço, avalie as seguintes perguntas:

1. **A consistência precisa ser imediata (ACID) ou pode ser eventual (BASE)?**
   - Se duas operações exigem transação distribuída em 2 fases (*two-phase commit*), elas provavelmente pertencem ao **mesmo** contexto/serviço. Se puderem ser resolvidas com consistência eventual (ex.: via padrão Saga), podem ser serviços separados.
2. **Qual é o vetor de mudança e escalabilidade?**
   - Se uma parte do sistema muda toda semana por demanda regulatória e outra quase nunca muda, isolá-las reduz risco de regressão.
   - Se o módulo de cálculo de frete recebe 100x mais tráfego que o módulo de faturamento, separá-los traz ganhos de escalabilidade granular.
3. **Qual é o alinhamento com a Lei de Conway?**
   - *A arquitetura do sistema espelha a estrutura de comunicação da organização.*

○ Um Bounded Context deve ter um **único time responsável** por ele. Evite criar serviços que demandem que múltiplos times façam commits concorrentes para entregar uma funcionalidade simples.

---

6. Estratégia Prática de Extração: O Padrão Strangler Fig (Estrangulamento)

Não tente quebrar o monólito todo de uma vez (*Big Bang rewrite*). O caminho mais seguro é incremental:

1. **Escolha um primeiro candidato:** Idealmente um subdomínio de suporte ou genérico, com fronteiras claras, pouco acoplamento e baixo risco ao negócio (ex.: módulo de notificações ou geração de relatórios).
2. **Defina a fronteira e crie a ACL:** Isole o acesso a esse domínio no monólito através de interfaces.
3. **Implemente o novo microsserviço:** Construa com sua própria base de dados (*database-per-service*).
4. **Redirecione o tráfego gradualmente:** Use um API Gateway ou proxy reverso para rotear as novas requisições para o novo serviço.
5. **Deprecie o código no monólito:** Remova o código legado após validação em produção.

Exemplo Mini e-commerce

Parte 1: Versão monolítica

```
// Cliente.java
@Entity
public class Cliente {
    @Id @GeneratedValue
    private Long id;
    private String nome;
    private String email;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}

// Produto.java
@Entity
public class Produto {
    @Id @GeneratedValue
    private Long id;
    private String nome;
    private BigDecimal preco;
    private int estoque;
```

```
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public BigDecimal getPreco() { return preco; }
    public void setPreco(BigDecimal preco) { this.preco = preco; }
    public int getEstoque() { return estoque; }
    public void setEstoque(int estoque) { this.estoque = estoque; }
}

// Pedido.java (note os @ManyToOne: acoplamento direto de dados)
@Entity
public class Pedido {
    @Id @GeneratedValue
    private Long id;
    @ManyToOne
    private Cliente cliente;
    @ManyToOne
    private Produto produto;
    private int quantidade;
    private BigDecimal total;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Cliente getCliente() { return cliente; }
    public void setCliente(Cliente cliente) { this.cliente = cliente; }
    public Produto getProduto() { return produto; }
```

```
    public void setProduto(Produto produto) { this.produto = produto; }
    public int getQuantidade() { return quantidade; }
    public void setQuantidade(int quantidade) { this.quantidade = quantidade; }
    public BigDecimal getTotal() { return total; }
    public void setTotal(BigDecimal total) { this.total = total; }
}

// Repositórios
public interface ClienteRepository extends JpaRepository<Cliente, Long> {}
public interface ProdutoRepository extends JpaRepository<Produto, Long> {}
public interface PedidoRepository extends JpaRepository<Pedido, Long> {}

// PedidoService.java
@Service
public class PedidoService {
    private final PedidoRepository pedidoRepo;
    private final ClienteRepository clienteRepo;
    private final ProdutoRepository produtoRepo;

    public PedidoService(PedidoRepository pedidoRepo,
                         ClienteRepository clienteRepo,
                         ProdutoRepository produtoRepo) {
        this.pedidoRepo = pedidoRepo;
        this.clienteRepo = clienteRepo;
        this.produtoRepo = produtoRepo;
    }
```

```
    @Transactional // uma transação ACID cobre tudo
    public Pedido criar(Long clienteId, Long produtoId, int qtd) {
        Cliente cliente = clienteRepo.findById(clienteId)
                .orElseThrow(() -> new IllegalArgumentException("Cliente não encontrado"));
        Produto produto = produtoRepo.findById(produtoId)
                .orElseThrow(() -> new IllegalArgumentException("Produto não encontrado"));

        if (produto.getEstoque() < qtd)
            throw new IllegalStateException("Estoque insuficiente");

        produto.setEstoque(produto.getEstoque() - qtd);

        Pedido p = new Pedido();
        p.setCliente(cliente);
        p.setProduto(produto);
        p.setQuantidade(qtd);
        p.setTotal(produto.getPreco().multiply(BigDecimal.valueOf(qtd)));
        return pedidoRepo.save(p);
    }
}

// Controllers
@RestController
@RequestMapping("/clientes")
public class ClienteController {
    private final ClienteRepository repo;
```

```
    public ClienteController(ClienteRepository repo) { this.repo = repo; }

    @PostMapping
    public Cliente criar(@RequestBody Cliente c) { return repo.save(c); }

    @GetMapping("/{id}")
    public Cliente buscar(@PathVariable Long id) {
        return repo.findById(id).orElseThrow();
    }
}

@RestController
@RequestMapping("/produtos")
public class ProdutoController {
    private final ProdutoRepository repo;

    public ProdutoController(ProdutoRepository repo) { this.repo = repo; }

    @PostMapping
    public Produto criar(@RequestBody Produto p) { return repo.save(p); }

    @GetMapping("/{id}")
    public Produto buscar(@PathVariable Long id) {
        return repo.findById(id).orElseThrow();
    }
}
```

```
@RestController

@RequestMapping("/pedidos")

public class PedidoController {

    private final PedidoService service;


    public record NovoPedido(Long clienteId, Long produtoId, int quantidade) {}


    public PedidoController(PedidoService service) { this.service = service; }


    @PostMapping

    public Pedido criar(@RequestBody NovoPedido r) {

        return service.criar(r.clienteId(), r.produtoId(), r.quantidade());

    }

}
```

Parte 2: Passo a passo da conversão

Identificar os Bounded Contexts: Cliente, Produto/Estoque e Pedido. Os acoplamentos estão em PedidoService (3 repositórios) e nos @ManyToOne de Pedido.

Definir serviços e donos de dados: cliente-service (8081), produto-service (8082), pedido-service (8083), cada um com seu banco.

Quebrar dependências de dados: @ManyToOne vira referência por ID (clienteId, produtoId).

Trocar chamadas de método por REST: clienteRepo.findById() vira GET no cliente-service. A baixa de estoque vira endpoint do produto-service.

Tratar a perda da transação única: consistência eventual com compensação (repor estoque). Em produção, use Saga.

Configurar cada serviço: application.properties com porta e banco próprios.

Parte 3: Versão com microsserviços

1) cliente-service (porta 8081)

```
server.port=8081
spring.datasource.url=jdbc:h2:mem:clientedb
spring.jpa.hibernate.ddl-auto=update

@Entity
public class Cliente {
    @Id @GeneratedValue
    private Long id;
    private String nome;
    private String email;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}

public interface ClienteRepository extends JpaRepository<Cliente, Long> {}

@RestController
@RequestMapping("/clientes")
public class ClienteController {
    private final ClienteRepository repo;

    public ClienteController(ClienteRepository repo) { this.repo = repo; }
```

```
    @PostMapping
    public Cliente criar(@RequestBody Cliente c) { return repo.save(c); }

    @GetMapping("/{id}")
    public ResponseEntity<Cliente> buscar(@PathVariable Long id) {
        return repo.findById(id).map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
}
```

2) produto-service (porta 8082)

```
server.port=8082
spring.datasource.url=jdbc:h2:mem:produtodb
spring.jpa.hibernate.ddl-auto=update

@Entity
public class Produto {
    @Id @GeneratedValue
    private Long id;
    private String nome;
    private BigDecimal preco;
    private int estoque;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNome() { return nome; }
```

```
    public void setNome(String nome) { this.nome = nome; }
    public BigDecimal getPreco() { return preco; }
    public void setPreco(BigDecimal preco) { this.preco = preco; }
    public int getEstoque() { return estoque; }
    public void setEstoque(int estoque) { this.estoque = estoque; }
}

public interface ProdutoRepository extends JpaRepository<Produto, Long> {}

@RestController
@RequestMapping("/produtos")
public class ProdutoController {
    private final ProdutoRepository repo;

    public ProdutoController(ProdutoRepository repo) { this.repo = repo; }

    @PostMapping
    public Produto criar(@RequestBody Produto p) { return repo.save(p); }

    @GetMapping("/{id}")
    public ResponseEntity<Produto> buscar(@PathVariable Long id) {
        return repo.findById(id).map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping("/{id}/baixa-estoque")
    @Transactional
```

```
    public ResponseEntity<Produto> baixar(@PathVariable Long id, @RequestParam int quantidade) {
        Produto p = repo.findById(id).orElse(null);
        if (p == null) return ResponseEntity.notFound().build();
        if (p.getEstoque() < quantidade) return ResponseEntity.status(409).build();
        p.setEstoque(p.getEstoque() - quantidade);
        return ResponseEntity.ok(p);
    }

    @PostMapping("/{id}/repor-estoque") // usado na compensação
    @Transactional
    public ResponseEntity<Void> repor(@PathVariable Long id, @RequestParam int quantidade) {
        Produto p = repo.findById(id).orElseThrow();
        p.setEstoque(p.getEstoque() + quantidade);
        return ResponseEntity.ok().build();
    }
}
```

3) pedido-service (porta 8083)

```
server.port=8083
spring.datasource.url=jdbc:h2:mem:pedidodb
spring.jpa.hibernate.ddl-auto=update
cliente.url=http://localhost:8081
produto.url=http://localhost:8082

// Apenas IDs, sem @ManyToOne
@Entity
public class Pedido {
```

```
    @Id @GeneratedValue
    private Long id;
    private Long clienteId;
    private Long produtoId;
    private int quantidade;
    private BigDecimal total;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Long getClienteId() { return clienteId; }
    public void setClienteId(Long clienteId) { this.clienteId = clienteId; }
    public Long getProdutoId() { return produtoId; }
    public void setProdutoId(Long produtoId) { this.produtoId = produtoId; }
    public int getQuantidade() { return quantidade; }
    public void setQuantidade(int quantidade) { this.quantidade = quantidade; }
    public BigDecimal getTotal() { return total; }
    public void setTotal(BigDecimal total) { this.total = total; }
}

public interface PedidoRepository extends JpaRepository<Pedido, Long> {}

// DTOs locais
public record ClienteDTO(Long id, String nome) {}
public record ProdutoDTO(Long id, String nome, BigDecimal preco, int estoque) {}

@Configuration
public class RestConfig {
```

```
    @Bean
    RestClient clienteClient(@Value("${cliente.url}") String url) {
        return RestClient.builder().baseUrl(url).build();
    }

    @Bean
    RestClient produtoClient(@Value("${produto.url}") String url) {
        return RestClient.builder().baseUrl(url).build();
    }
}

@Service
public class PedidoService {
    private final PedidoRepository pedidoRepo;
    private final RestClient clienteClient;
    private final RestClient produtoClient;

    public PedidoService(PedidoRepository pedidoRepo,
                         @Qualifier("clienteClient") RestClient clienteClient,
                         @Qualifier("produtoClient") RestClient produtoClient) {
        this.pedidoRepo = pedidoRepo;
        this.clienteClient = clienteClient;
        this.produtoClient = produtoClient;
    }

    public Pedido criar(Long clienteId, Long produtoId, int qtd) {
        // 1. Valida cliente
```

```
try {
    clienteClient.get().uri("/clientes/{id}", clienteId)
            .retrieve().body(ClienteDTO.class);
} catch (HttpClientErrorException.NotFound e) {
    throw new IllegalArgumentException("Cliente não encontrado");
}

// 2. Busca produto (preço)
ProdutoDTO produto;
try {
    produto = produtoClient.get().uri("/produtos/{id}", produtoId)
            .retrieve().body(ProdutoDTO.class);
} catch (HttpClientErrorException.NotFound e) {
    throw new IllegalArgumentException("Produto não encontrado");
}

// 3. Baixa estoque
try {
    produtoClient.post()
            .uri("/produtos/{id}/baixa-estoque?quantidade={q}", produtoId, qtd)
            .retrieve().toBodilessEntity();
} catch (HttpClientErrorException.Conflict e) {
    throw new IllegalStateException("Estoque insuficiente");
}

// 4. Salva o pedido; se falhar, compensa o estoque
try {
```

```
            Pedido p = new Pedido();
            p.setClienteId(clienteId);
            p.setProdutoId(produtoId);
            p.setQuantidade(qtd);
            p.setTotal(produto.preco().multiply(BigDecimal.valueOf(qtd)));
            return pedidoRepo.save(p);
        } catch (RuntimeException e) {
            produtoClient.post()
                    .uri("/produtos/{id}/repor-estoque?quantidade={q}", produtoId, qtd)
                    .retrieve().toBodilessEntity();
            throw e;
        }
    }
}

@RestController
@RequestMapping("/pedidos")
public class PedidoController {
    private final PedidoService service;

    public record NovoPedido(Long clienteId, Long produtoId, int quantidade) {}

    public PedidoController(PedidoService service) { this.service = service; }

    @PostMapping
    public Pedido criar(@RequestBody NovoPedido r) {
        return service.criar(r.clienteId(), r.produtoId(), r.quantidade());
```

```
    }
}
```

Parte 4: Testando

```
curl -X POST localhost:8081/clientes -H "Content-Type: application/json" \
  -d '{"nome":"Ana","email":"ana@email.com"}'

curl -X POST localhost:8082/produtos -H "Content-Type: application/json" \
  -d '{"nome":"Teclado","preco":150.00,"estoque":10}'

curl -X POST localhost:8083/pedidos -H "Content-Type: application/json" \
  -d '{"clienteId":1,"produtoId":1,"quantidade":2}'
```

O pedido retorna total = 300.00, e GET localhost:8082/produtos/1 mostra estoque 8.